

CoppeliaSim Tutorial 7

懸垂二足モデルと骨盤への外力実験

これまでのチュートリアルで学んだ知識を活用して、**吊り下げ式の二足モデル**を作成します。このモデルでは**骨盤部に外力を加え**、その力がどのように下肢に伝わるかを観察します。

🎯 目的

- **体幹 + 骨盤 + 両脚** の懸垂二足モデルを自作する
- 各部位の質量・ジョイント設定を理解する
- **骨盤への外力**が下肢運動にどう影響するか観察する
- 力の伝達と下肢の揺れを観察する

🔧 使用環境

- CoppeliaSim (Bullet 物理エンジン)
- **前提:** Tutorial 1〜6 を完了していること

1. モデル構造の設計

1.1 構成要素



1.2 各部位のサイズ (参考値)

部位	形状	サイズ (m)	質量 (kg)	動的
Trunk	Cuboid	0.15 × 0.10 × 0.15	-	OFF

部位	形状	サイズ (m)	質量 (kg)	動的
Pelvis	Cuboid	0.20 × 0.10 × 0.08	2.0	ON
Thigh	Cylinder	半径 0.03, 長さ 0.30	1.0	ON
Shank	Cylinder	半径 0.025, 長さ 0.30	0.7	ON
Foot	Cuboid	0.06 × 0.15 × 0.03	0.3	ON

2. 体幹と骨盤を作成

2.1 Trunk（体幹・固定用）の作成

1. **Add → Primitive shape → Cuboid**
2. サイズ: X=0.15, Y=0.10, Z=0.15
3. **Create dynamic and respondable shape** にチェック
4. **OK** をクリック
5. オブジェクトを **ダブルクリック** してプロパティを開く
6. **Show dynamic properties dialog** をクリック
7. **Body is dynamic** のチェックを**外す**（静的オブジェクトにする）
8. **Body is respondable** のチェックも**外す**（衝突しないようにする）
9. 名前を **Trunk** に変更
10. 位置を調整: **Z = 0.84**

2.2 PelvisJoint（球関節）の作成

1. **Add → Joint → Spherical**
2. 名前を **PelvisJoint** に変更
3. 位置を調整: **Z = 0.76**（Trunkの下端付近）
4. **PelvisJoint** を **Trunk** の子にする

2.3 Pelvis（骨盤）の作成

1. **Add → Primitive shape → Cuboid**
2. サイズ: X=0.20, Y=0.10, Z=0.08
3. **Create dynamic and respondable shape** にチェック
4. **OK** をクリック
5. オブジェクトを **ダブルクリック** してプロパティを開く
6. **Show dynamic properties dialog** をクリック
7. **Body is dynamic** が**ON**になっていることを確認（動的オブジェクト）
8. **Body is respondable** のチェックを**外す**（脚やTrunkとの衝突防止）
9. **Mass = 2.0 kg** に設定
10. 名前を **Pelvis** に変更
11. 位置を調整: **Z = 0.72**
12. **Pelvis** を **PelvisJoint** の子にする

💡 この構造により、Trunkは固定され、Pelvisは球関節で吊り下げられた状態になります。外力を加えるとPelvisが揺れ、その動きが下肢に伝わります。

3. 右脚を作成

3.1 右股関節 (RHip)

1. **Add → Joint → Revolute**
2. 名前を **RHip** に変更
3. **Orientation** タブで回転を設定: **Alpha=0, Beta=90, Gamma=0**
 - これでジョイントの回転軸 (Z軸) がX軸方向 (前後屈伸方向) に向きます。
4. **Position** タブで位置を **X = 0.05, Z = 0.68** に設定 (大腿の上端付近)

3.2 右大腿 (RThigh)

1. **Add → Primitive shape → Cylinder**
2. サイズ: X=0.06, Y=0.06, Z=0.30
 - 注: Cylinder (円柱) の場合、XとYのサイズは連動します。片方を変更すると自動的にもう一方も同じ値になります。
3. **Create dynamic and respondable shape** にチェック
4. **OK** をクリック
5. **Double click** the shape to open properties.
6. **Show dynamic properties dialog** -> **Body is respondable** のチェックを**外す** (骨盤や膝との干渉防止)
7. **Mass = 1.0 kg** に設定
8. 名前を **RThigh** に変更
9. **Position** タブで位置を **X = 0.05, Z = 0.53** に設定

3.3 階層構造を構築

1. Scene hierarchy で以下の順にドラッグ&ドロップ:

- **RThigh** を **RHip** の子にする
- **RHip** を **Pelvis** の子にする

△ 注意: 回転 (Orientation) の設定は、**階層化する前**に行うのが安全です。階層化後に親を回転させると、子も一緒に回転してしまいます。

もし一緒に回転してしまったら:

1. Scene hierarchy で子オブジェクト (例: **RThigh**) をドラッグし、何もないところ (World直下) にドロップして**階層を解除**します。
2. 親 (**RHip**) の回転を修正します。
3. 再度、子を親にドラッグ&ドロップします。

2. 位置が正しければ、骨盤の下に大腿が配置されているはずです。

3.4 右下腿 (RShank) と右膝関節 (RKnee)

RKnee (膝関節) を作成

1. **Add → Joint → Revolute**
2. 名前を **RKnee** に変更
3. **Orientation** タブで回転を設定: **Alpha=0, Beta=90, Gamma=0**

4. **Position** タブで位置を **X = 0.05, Z = 0.38** に設定（大腿の下端付近）

RShank（下腿）を作成

1. **Add → Primitive shape → Cylinder**
2. サイズ: X=0.05, Y=0.05, Z=0.30
3. **Create dynamic and respondable shape** にチェック
4. **OK** をクリック
5. **Double click → Dynamic properties → Body is respondable** のチェックを外す
6. **Mass = 0.7 kg** に設定
7. 名前を **RShank** に変更
8. **Position** タブで位置を **X = 0.05, Z = 0.23** に設定

階層構造を構築

1. **RShank** を **RKnee** の子にする
2. **RKnee** を **RThigh** の子にする

3.5 右足部（RFoot）と右足首関節（RAnkle）

RAnkle（足首関節）を作成

1. **Add → Joint → Revolute**
2. 名前を **RAnkle** に変更
3. **Orientation** タブで回転を設定: **Alpha=0, Beta=90, Gamma=0**
4. **Position** タブで位置を **X = 0.05, Z = 0.08** に設定（下腿の下端付近）

RFoot（足部）を作成

1. **Add → Primitive shape → Cuboid**
2. サイズ: X=0.06, Y=0.15, Z=0.03（Y方向が前後）
3. **Create dynamic and respondable shape** にチェック
4. **OK** をクリック
5. **Mass = 0.3 kg** に設定
6. 名前を **RFoot** に変更
7. **Position** タブで位置を **X = 0.05, Y = 0.05, Z = 0.065** に設定
 - X = 0.05: 右側オフセット
 - Y = 0.05: 前方オフセット
 - Z = 0.065: 床から5cm浮いた位置（吊り下げ状態）

階層構造を構築

1. **RFoot** を **RAnkle** の子にする
2. **RAnkle** を **RShank** の子にする

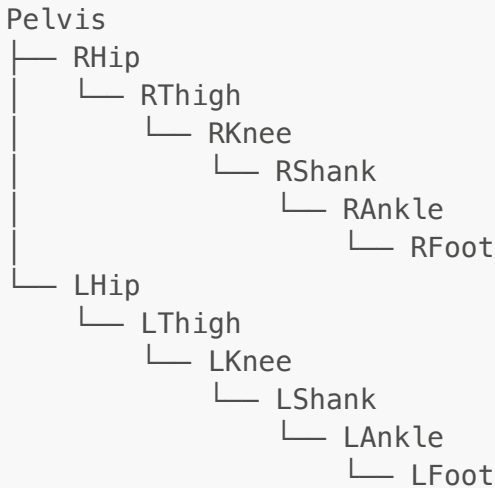
4. 左脚を作成

4.1 右脚をコピー

1. **RHip** を選択
2. **Ctrl+C** でコピー、**Ctrl+V** でペースト
3. ペーストしたオブジェクトを **LHip** にリネーム
4. 子オブジェクトも L で始まる名前に変更 (LThigh, LKnee, LShank, LAnkle, LFoot)
5. **LHip** を選択し、**Position** タブで **X = -0.05** に設定 (左側に配置)

💡 階層構造でコピーした場合、親 (LHip) の位置を変更すると子も一緒に移動します。

4.2 階層構造の確認



5. 地面との接触設定

5.1 足部の衝突設定

1. **RFoot** をダブルクリックしてプロパティを開く
2. **Respondable** がONになっていることを確認
3. **LFoot** も同様に確認

6. ジョイント設定

6.1 各関節のモード設定

すべての関節に対して:

1. ジョイントをダブルクリック
2. **Mode** が **Dynamic mode** になっていることを確認 (物理演算で自由に動く)

💡 Bulletエンジンにはジョイントの Damping 設定がありません。動きを抑制したい場合は MuJoCo や Vortex エンジンを検討してください。

6.2 関節角度の制限 (オプション)

膝が逆に曲がらないように制限を設定:

1. RKnee と LKnee を選択
2. **Position is cyclic** のチェックを外す
3. **Pos. min [deg] = -120, Pos. range [deg] = 120** に設定

7. 骨盤への外力スクリプト

7.1 スクリプトを追加

1. Pelvis を右クリック
2. **Add → Script → Simulation script → Non-threaded → Lua**

7.2 スクリプト内容

このスクリプトでは、以下の条件で外力を加えるように設定します：

- **加える場所**: 骨盤 (Pelvis) の中心
- **力の大きさ**: 15.0 N
- **力の方向**: Y軸方向 (横方向)
- **加える時間**: 実験開始 2.0秒後 から 0.3秒間

```
function sysCall_init()
    pelvisHandle = sim.getObject('/Pelvis')

    -- 無重力状態にする (足部の垂れ下がり防止)
    -- sim.setFloatParam(sim.floatparam_gravity_z, 0) -- 古い記述
    sim.setArrayParameter(sim.arrayparam_gravity, {0, 0, 0})

    print("Zero Gravity Mode: ON (Gravity set to 0,0,0)")

    forceDelay = 2.0          -- 2秒後に力を加え始める
    forceDuration = 0.3       -- 0.3秒間力を加える
    forceMagnitude = 15.0     -- 15ニュートン
    forceStarted = false
    forceEnded = false

    print("=== Pelvis Force Demo Ready ===")
    print("Force will be applied at t=" .. forceDelay .. " seconds")
end

function sysCall_actuation()
    local t = sim.getSimulationTime()

    if t >= forceDelay and t < forceDelay + forceDuration then
        if not forceStarted then
            forceStarted = true
            print("Force started - pushing pelvis sideways!")
        end

        -- 横方向 (Y軸) に力を加える
        local force = {0, forceMagnitude, 0}
        local position = {0, 0, 0} -- 骨盤の中心
```

```

        sim.addForce(pelvisHandle, position, force)

elseif t >= forceDelay + forceDuration and not forceEnded then
    forceEnded = true
    print("Force ended!")
end
end
end

```

8. シミュレーション実行

8.1 実行前の確認

- ☐ すべてのオブジェクトが **Dynamic** になっている
- ☐ 足部が床より少し上にある（埋まっていない）
- ☐ ジョイントが正しく接続されている

8.2 実行手順

1. **Simulation → Toggle real-time mode** を選択してONにする
2. ▶ **Start simulation**
3. モデルが立位を保とうとする様子を観察
4. **2秒後** に骨盤が横から押される
5. 下肢の反応を観察

8.3 観察ポイント

- ☐ 骨盤が押されたとき、各関節はどう動くか？
- ☐ 足部は床から離れるか？
- ☐ どの方向に倒れるか？

9. 実験：パラメータを変える

実験1: 力の大きさ

```

forceMagnitude = 5.0    -- 弱い力
forceMagnitude = 20.0   -- 強い力
forceMagnitude = 30.0   -- 転倒する力

```

実験2: 力の方向

```

local force = {forceMagnitude, 0, 0}  -- 前後方向
local force = {0, forceMagnitude, 0}  -- 横方向
local force = {0, 0, forceMagnitude}  -- 上向き

```

実験3: 力を加える位置

```
local position = {0, 0, 0.04}  -- 骨盤の上部
local position = {0, 0, -0.04} -- 骨盤の下部
```

 **考察:** 力の方向や加える位置によって、下肢の反応はどう変わりますか？

10. トラブルシューティング

症状	チェック項目
モデルが落下する	足部が床より上にあるか確認
関節が動かない	ジョイントモードが Passive になっているか
すぐに転倒する	Damping を追加する
力が効かない	オブジェクトが Dynamic か確認
オブジェクトが分離する	階層構造が正しいか確認

11. シーンを保存

1. **File** → **Save scene as...**
2. ファイル名: **SimpleBiped_Force.ttt**

 物理的な補足

骨盤への外力と下肢運動

骨盤に外力が加わると:

1. **骨盤が傾く** → 股関節トルクが発生
2. **股関節トルク** → 大腿が動く
3. **大腿の動き** → 膝関節トルクが発生
4. **連鎖的に** 下肢全体が反応

これは人間の姿勢反応と同様のメカニズムです。

力の伝達経路

外力 → 骨盤 → 股関節 → 大腿 → 膝関節 → 下腿 → 足首関節 → 足部 → 床反力

 次へのステップ (オプション: 7B)

立位モデルに**簡単な周期的な脚振り**を追加して、動作中の外力実験を行うことも可能です。

興味があれば、以下をスクリプトに追加:

```
-- 簡易的な脚振り（股関節のみ）
function sysCall_actuation()
    local t = sim.getSimulationTime()

    -- 脚振りの周期（1秒周期、振幅15度）
    local amplitude = 15 * math.pi / 180 -- 15度
    local frequency = 1.0 -- 1Hz

    local rHipAngle = amplitude * math.sin(2 * math.pi * frequency * t)
    local lHipAngle = -rHipAngle -- 左右で逆位相

    -- 股関節に位置指令（Passive→Forceモードに変更が必要）
    -- sim.setJointTargetPosition(rHipHandle, rHipAngle)
    -- sim.setJointTargetPosition(lHipHandle, lHipAngle)

    -- 外力処理（既存のコード）
    -- ...
end
```



復習問題

1. 骨盤への外力が下肢に伝わる経路を説明してください
2. 力を加える位置（骨盤の上部 vs 下部）で反応が異なる理由は？
3. 関節の Damping を大きくすると、外力への反応はどう変わりますか？
4. このモデルとNAOの歩行モデルの違いは何ですか？